Maturitätsarbeit an der Kantonsschule Zürich Nord

Optimierung einer 3D Rendering Engine in Godot für ein First Person Shooter Computerspiel

Verfasser/in:	**Tomas Rivera**
Klasse:	M6e
Betreuungsperson:	Andreas Steingötter
Abgabedatum:	Zürich, 23. November 2026

Inhaltsverzeichnis

Motivation 1
1 Einleitung 1
1.1 Relevanz und Fragestellung 1
1.2 Wahl der Entwicklungsumgebung (Godot) 1
2 Die Rendering-Algorithmen 2
2.1 Wireframe-Rendering 2
2.2 Raycasting 4
2.3 Voxel-Ray-Traversal 5
3 Umsetzung in Godot 5
3.1 Software Architektur und Implementierung 5
3.2 Testaufbau 6
3.2.1 Aufbau Szenen in Blender 6
3.2.2 Datenstrukturen der Szenen 6
3.3 Simulation 7
4 Ergebnisse 7
5 Interpretation 8
5.1 Vergleichende Analyse der Algorithmen 8
5.2 Limitationen der Godot Implementierung 9
6 Zusammenfassung und Schlusswort 9
Quellenverzeichnis 10

Motivation

Ich interessiere mich für die Entwicklung von Videospielen und fand schon immer, dass man ein Programm nicht richtig versteht bevor man die einzelnen Teile analysiert oder sogar selbst programmiert hat. Ausserdem hat es mich schon immer gewundert wie ein Computer eine dreidimensionale Szene nimmt und sie dann auf zwei Dimensionen projizieren kann.

Indem ich lerne wie die Sachen funktionieren die ich täglich nutze kann ich selber besser mit ihnen arbeiten wenn ich ein Spiel programmiere und verstehe was hinter ihnen steckt.

1 Einleitung

1.1 Relevanz und Fragestellung

Bei modernen Videospielen, die kompetitiv gespielt werden und höchste Präzision verlangen, besonders bei sogenannte «First-Person-Shooter» aus der Ich-Perspektive wird die Optimierung von allen Teilen des Programmes immer wichtiger. Wenige Millisekunden könnte dazu führen, dass man das ganze Spiel verliert. Die Geschwindigkeit eines Spieles wird in Bildern pro Sekunde gemessen, sogenannte «Frames per Second» oder FPS.

Einer der wichtigsten Teile dieser Spiele ist der Renderer, also der Teil des Programms, der alle dreidimensionalen Objekte in einer Szene verarbeitet und dann auf ein zweidimensionales Bild projiziert, das dann auf dem Bildschirm gezeigt wird. Ohne den Renderer würde man nichts sehen.

Das Ziel meiner Maturitätsarbeit ist die Leistung von verschiedenen Rendering-Algorithmen zu messen, indem ich selber drei technisch völlig unterschiedliche Rendering-Methoden programmiere. Sie sind:

- **Wireframe-Rendering:** Ein Algorithmus, der Objekte wie Gitter darstellt und mit einer einfachen Formel sie auf den Bildschirm projiziert.
- **Raycasting:** Ein Algorithmus, der für jedes Pixel auf dem Bildschirm einen Strahl schickt und sieht, ob es ein Objekt trifft und wie weit es ist.
- **Voxel-Ray-Traversal:** Ein Algorithmus, der ähnlich zu Raycasting funktioniert, jedoch nur in einen Raum aus Blöcken oder sogenannten «Voxels».

Wie genau diese Algorithmen funktionieren wird im nächsten Kapitel erklärt. Danach werde ich sie mithilfe von Beispielszenen miteinander vergleichen, die Daten analysieren und ihre Vor- und Nachteile erörtern.

Daraus ergibt sich die zentrale Frage meiner Arbeit:

Wie verhalten sich diese drei Rendering-Methoden bezüglich Rechenzeit und Effizienz, und welche Vor- und Nachteile bieten sie für den praktischen Einsatz bei welchen Fällen in einem Computerspiel?

1.2 Wahl der Entwicklungsumgebung (Godot)

Um die Renderer zu programmieren wählte ich die Spiel-Engine Godot aus. Godot ist die meist benutzte quelloffene Spiel-Engine und ist im Vergleich zu anderen einfach zu programmieren. Als Programmiersprache wird GDScript in Godot genutzt. GDScript ist sehr ähnlich zu Python und ist deswegen leicht zu lesen und einfach zum lernen.

Da es eine Spiel-Engine ist, hat es auch eingebaute Funktionen wie zum Beispiel `draw_polyline()` um mehrere Punkte auf den Bildschirm zu verbinden. Godot ist gemacht für zweidimensionale und dreidimensionale Spiele, ich nutze aber bewusst nur die Funktionen für zwei Dimensionen.

Wie diese Architektur in Godot konkret umgesetzt wurde und wie die Test Szenen aufgebaut sind, wird in Kapitel 3 erläutert.

2 Die Rendering-Algorithmen

Bei der Wahl der Rendering-Algorithmen versuchte ich drei möglichst verschiedene Algorithmen auszuwählen.

Alle drei nutzen verschiedene Datenstrukturen um Objekte auf zwei Dimensionen zu projizieren und alle Drei machen das auf verschieden Arten.

2.1 Wireframe-Rendering

Das Wireframe-Rendering ist die älteste Methode der 3D-Grafik. Objekte werden nur mit ihren Kanten gezeichnet und sehen deswegen aus, als wären sie aus Draht.

Der erste Schritt ist es, alle Koordinaten relativ zum Spieler, beziehungsweise zur Kamera zu berechnen. Man macht sie zum Nullpunkt des Raumes und die Position der Kamera wird von der Position des Objektes subtrahiert.

Die Kamera blickt zu Beginn entlang der Z-Achse, also in den Richtungsvektor $\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$ Um die Kamera zu rotieren muss man die zweidimensionale Bewegung der Maus in drei Dimensionen umwandeln. Dies geschieht in zwei Schritten:

1. Bewegt man die Maus horizontal, wird der Richtungsvektor um die Y-Achse des Raumes nach links oder rechts rotiert. Dabei verändern sich die X- und Z-Komponenten des Richtungsvektors.
2. Bewegt man die Maus vertikal, wird die Kamera um ihre X-Achse rotiert, nach oben oder unten. Dabei verändern sich die Y- und Z-Komponenten des Richtungsvektors.

Da bei beiden Schritten die Z-Komponente angepasst wird, lässt sich der Richtungsvektor über diese zwei einfachen Mausbewegungen frei in allen drei Raumdimensionen ausrichten.

Weil die Kamera aber nicht nur aus einer Blickrichtung besteht, sondern auch wissen muss, wo oben und rechts ist, fasst das Programm diese Achsen in einer sogenannten Rotationsmatrix zusammen.

Diese Drehung wird danach invertiert, da es so ist, dass wenn man nach rechts schaut sich alles nach links bewegt und wenn man nach oben schaut sich alles nach unten bewegt. Alle Objekte in der Szene werden dann mit dieser invertierten Matrix multipliziert.

Danach kann man mit dieser Formel diese dreidimensionalen Punkte umrechnen:

$$x' = \frac{x}{z} \qquad y' = \frac{y}{z} \tag{1}$$

x' und y' sind dann zweidimensionale Koordinaten die auf dem Bildschirm gezeigt werden können. Das Prinzip der Formel ist die perspektivische Umrechnung, man teilt die Koordinaten x und y durch den Abstand

z Je weiter ein Objekt ist, desto grösser ist z und das Ergebnis wird kleiner, also erscheinen Objekte die weiter weg sind kleiner.

Da x' und y' Werte im Bereich von -1 bis $+1$ liefern, müssen sie abschliessend auf die tatsächliche Fensterauflösung skaliert und um die halbe Bildschirmbreite beziehungsweise Bildschirmhöhe verschoben werden, um den Mittelpunkt des Monitors zu treffen. Erst danach verbindet Godots Zeichenfunktion `draw_polyline()` die berechneten Pixelpunkte.

Es gibt aber zwei Fälle, bei denen die Formeln nicht funktionieren:

- Wenn ein Objekt hinter dem Spieler erschient, zum Beispiel falls es $(1, 1, -1)$ hinter dem Spieler ist, dann wird nach Anwendung der Formel das Objekt auf den Bildschirm bei den Koordinaten $(-1, -1)$ sein. Das heisst das Objekte mit negativem Z-Wert, also Objekte hinter dem Spieler, invertiert auf dem Bildschirm gezeigt werden.
- Ein Objekt kann nicht die Z-Koordinate 0 haben, da eine Division durch 0 nie möglich ist

Um diese Probleme zu lösen, überprüft das Programm zuerst ob alle Eckpunkte eines Objektes hinter dem Spieler sind oder vor dem Spieler. Wie schon vorher erwähnt blickt die Kamera in die relative Z-Richtung, dass bedeutet das alle Punkte mit positiven Z-Wert vor den Spieler und alle Punkte mit negativen Z-Wert hinter den Spieler sind. Daraus lässt sich kurz vor der Kamera eine Schnittebene definieren, die sogenannte Near-Clipping-Plane, die im Code bei $Z = 0.1$ liegt. Sie verhindert, dass Punkte mit $Z \leq 0$ überhaupt in die Projektionsformel gelangen.

Falls nicht alle Eckpunkte vor oder hinter dem Spieler sind, also gemischt sind, dann müssen alle Flächen des Objekts entlang der Near-Clipping-Plane so «geschnitten» werden, dass es keine Eckpunkte mehr hinter dem Spieler gibt. Dieser Vorgang heisst «clipping». Der Sutherland-Hodgman-Algorithmus [1] ist dazu gemacht um mit den Flächen eines Objektes genau das zu machen.

Der Sutherland-Hodgman-Algorithmus geht alle Ecken einer Fläche durch und vergleicht sie mit der nächsten Ecke. Dabei wird eine Ecke immer als B und die nächste als A bezeichnet. Es gibt vier mögliche Fälle:

- Falls A und B sichtbar sind, dann wird B gespeichert.
- Falls nur A sichtbar ist, muss man berechnen ab welchen Punkt P die Kante AB nicht mehr sichtbar ist. Für das nutzt man lineare Interpolation, die mit der Variable t, dem sogenannten Gewicht, berechnen kann bei welchen Koordinaten ein Punkt zwischen zwei anderen Punkten liegt. $t = 1$ würde in diesen Fall bedeuten, dass der Punkt genau bei B liegt und $t = 0.5$ würde bedeuten dass der Punkt genau zwischen A und B liegt. Die Formel für das Gewicht ist $t = \frac{0.1 - A_z}{B_z - A_z}$. Um danach zu berechnen wo der Punkt liegt nutzt man die Formel $P = A + t(B - A)$. Zusammengefasst ist die Formel

$$P = A + \frac{0.1 - A_z}{B_z - A_z}(B - A) \tag{2}$$

  Der Punkt P wird dann gespeichert.
- Falls nur B sichtbar ist, wird die gleiche Formel wie beim letzten Fall angewendet. Der Punkt P wird gespeichert und es wird ausserdem Punkt B gespeichert. Dies ist so, weil die Formel B immer mit A vergleicht, also wenn B sichtbar ist wird B grundsätzlich gespeichert.
- Falls A und B nicht sichtbar sind, dann wird nichts gemacht.

2.2 Raycasting

Der Begriff Raycasting leitet sich aus dem Englischen ab und bedeutet das aussenden von Strahlen. Für jedes Pixel auf dem Bildschirm sendet der Algorithmus einen Strahl aus um zu sehen, wie weit ein Objekt vom Spieler ist.

Dafür muss ein Strahl mit seiner Richtung $\vec{\boldsymbol{d}}$ als Vektor und seinen Ursprung $\boldsymbol{o}$ als Koordinate definiert werden. Beide werden in Formeln später gebraucht.

Der Ursprung des Strahles ist die derzeitige Position der Kamera.

Die Richtung des Strahles wird berechnet, indem man die vier Ecken des Bildschirmes als Vektoren definiert, diese mit der Rotationsmatrix der Kamera multipliziert und dann je nach gewünschter Auflösung des Bildschirmes zwischen diesen Werten interpoliert, also alle Werte zwischen den vier Ecken ausrechnet. Je mehr Strahlen berechnet werden, desto höher ist die Auflösung.

Jetzt wo man alle Strahlen hat, fehlen nur noch die Objekte. Im Gegensatz zum Wireframe-Rendering werden Objekte nicht als Polygonen Flächen gegeben, sondern in Dreiecken.

Sie müssen so gegeben sein, da man für dreidimensionales Raycasting den Möller-Trumbore-Algorithmus [2] nutzt, der nur mit Dreiecken funktioniert. Er kann mit den Eckpunkten eines Dreiecks und den Ursprung und Richtung eines Strahles berechnen, ob der Strahl das Dreieck trifft und wie weit der Strahl laufen musste, um das Dreieck zu treffen.

Um zu verstehen wie der Algorithmus funktioniert, muss man zuerst baryzentrische Koordinaten verstehen. Man definiert zuerst zwei beliebige Kanten eines Dreiecks als Vektoren, zum Beispiel bei einem Dreieck ABC die Vektoren $\overrightarrow{AB}$ und $\overrightarrow{AC}$. Indem man $u * \overrightarrow{AB}$ und $v * \overrightarrow{AC}$ multipliziert kann man alle Punkte innerhalb und ausserhalb des Dreiecks mit u und v definieren.

Wenn $u < 0$ oder $v < 0$ oder $u + v > 1$, dann weiss man, dass ein Punkt ausserhalb des Dreiecks liegt.

Der Möller-Trumbore-Algorithmus nutzt genau diese baryzentrischen Koordinaten, um in vier Schritten zu überprüfen, ob ein Strahl ein Dreieck trifft.

Zuerst berechnet das Programm die beiden Kantenvektoren des Dreiecks $\overrightarrow{AB}$ und $\overrightarrow{AC}$. ausgehend vom ersten Eckpunkt A. Anschliessend wird über das Kreuzprodukt und das Skalarprodukt mit der Formel $\det = \overrightarrow{AB} \cdot \left(\vec{d} \times \overrightarrow{AC}\right)$ die Determinante berechnet. Diese Zahl beschreibt geometrisch, wie schräg der Strahl zur Dreiecksfläche steht. Ist der Wert sehr nahe an null, verläuft der Strahl parallel zur Fläche des Dreiecks und kann es niemals schneiden.

Es wird dafür eine winzige Schranke namens $\text{Epsilon} = 0.0000001$ definiert. Um Rundungsfehler des Computers zu erkennen schaut das Programm ob die Determinante zwischen $+\text{Epsilon}$ und $-\text{Epsilon}$ liegt, falls ja bricht die Funktion sofort ab, da kein Treffer möglich ist.

Ist der Strahl nicht parallel, berechnet der Algorithmus die erste baryzentrische Koordinate u. Die Formel dafür ist $\quad u = \frac{1}{\det} \cdot (o - A) \cdot \left(\vec{d} \times \overrightarrow{AC}\right)$.

Das Programm prüft:

- Ist $u < 0$ oder $u > 1$, verfehlt der Strahl bereits die erste Begrenzung des Dreiecks. Das Programm bricht ab.

Liegt u im gültigen Bereich, wird im nächsten Schritt die zweite baryzentrische Koordinate v ermittelt mit der Formel $\quad v = \frac{1}{\det} \cdot (o - A) \cdot \left(\overrightarrow{AB} \times \vec{d}\right)$. Hier folgt die Bedingung aus der Definition der baryzentrischen Koordinaten:

- Ist $v < 0$ oder ist die Summe aus $u + v > 1$, schiesst der Strahl über die dritte Kante des Dreiecks hinaus. Auch hier bricht das Programm direkt ab.

Hat der Strahl alle bisherigen Prüfungen bestanden, steht fest, dass er die Fläche des Dreiecks durchdringt. Als Letztes muss nun berechnet werden, wie weit der Treffpunkt von der Kamera entfernt ist mit der Formel Distanz $= \frac{1}{\det} \cdot (o - A) \cdot \left(\overrightarrow{AB} \times \overrightarrow{AC}\right)$. Ist diese Distanz grösser als Epsilon, liegt der Schnittpunkt im sichtbaren Bereich vor der Kamera. Ist die Distanz hingegen negativ, liegt das Dreieck hinter dem Spieler und wird ignoriert.

Da ein Strahl in einer kompletten Szene mehrere Dreiecke hintereinander durchdringen kann, speichert der Algorithmus für jeden Pixel den Treffer mit der geringsten Distanz ab.

Durch dieses schrittweise Aussortieren spart der Algorithmus viel Rechenzeit, da die aufwendigeren Rechnungen nur für jene Strahlen ausgeführt werden, die das Dreieck tatsächlich treffen können.

2.3 Voxel-Ray-Traversal

Der Voxel-Ray-Traversal-Algorithmus, auch bekannt als der Amanatides-Woo-Algorithmus [3], sendet ähnlich wie der Raycasting-Algorithmus einen Strahl für jeden Pixel des Bildschirms aus. Jedoch nutzt er nicht Dreiecke, sondern sogenannte volumetrische Pixels oder Voxels. Um also diesen Algorithmus zu nutzen muss man entweder alle Objekte von Anfang an als Voxels haben oder eine Szene zuerst in Voxels verwandeln.

(Voxelizer Erklärung hier)

Anstatt dass jeder ausgesendete Strahl bei allen Dreiecken der Szene überprüft ob es ein Dreieck trifft und dann berechnet wie weit das getroffene Dreieck ist, macht der Voxel-Ray-Traversal das viel effizienter.

Es berechnet alle Voxels des Gitters durch die der Strahl geht und für jeden durch den er verläuft, prüft er, ob er besetzt ist oder nicht. Falls ja, dann stoppt er, falls nein geht er weiter bis er auf einen besetzten Voxel trifft oder die maximal definierte Anzahl an Schritten erreicht hat.

Um die Strahlen zu berechnen wird die genau gleiche Funktion wie beim Raycasting-Algorithmus genutzt. Sie gibt einem für die gewünschte Auflösung für jeden Pixel die Richtung $\vec{\boldsymbol{d}}$ des Strahles. Der Ursprung $\boldsymbol{o}$ des Strahles ist weiterhin die Position der Kamera.

Als erstes muss die Kamera wissen in welchem Voxel des Gitters sie sich befindet. Dies wird gemacht indem die Kamera Position durch die Grösse der Voxel geteilt wird und danach abgerundet wird.

Es wird dann der Abstand berechnet, den der Strahl braucht um von einer ganzen x, y oder z-Koordinate zur anderen zu gehen. Dies wird mit der Formel $\Delta x = \left|\frac{1}{\vec{d}_x}\right| \quad \Delta y = \left|\frac{1}{\vec{d}_y}\right| \quad \Delta z = \left|\frac{1}{\vec{d}_z}\right|$ berechnet.

(restliche Erklärung fehlt noch)

3 Umsetzung in Godot

3.1 Software Architektur und Implementierung

Bei der Software Architektur entschied ich mich für einen Objekt-Orientierten Ansatz. Ich teilte meine Dateien so auf, dass ich eine Haupt-Datei hatte, die die Renderer kontrollieren kann. Diese Haupt-Datei war zuständig für die Kamera Bewegung und Kamera Rotation, für das weitergeben dieser Daten zu den Rendering-Algorithmen, den Input des Spielers und das Benchmarking der verschiedenen Rendering-Algorithmen. Die Dateien wurden mit einer Funktion als Instanz zu der Haupt-Datei angefügt.

Die Berechnungen für jeden Frame wurden mit der Godot-Funktion `process(delta)` gemacht. Delta ist die Zeit die zwischen jeden Frame vergangen ist. Jedes Mal, wenn ein neuer Frame erzeugt wird, ruft Godot automatisch die Funktion `process(delta)` auf. Alle Bewegungen, die zwischen den Frames berechnet werden, werden mit delta multipliziert, da die Zeit (in diesem Fall delta) mal die Geschwindigkeit gleich die Strecke ist.

In Godot stösst man dabei auf ein Problem, da Godot bei extrem niedrigen Bildwiederholraten unter 7 den delta Wert deckelt, damit die Physik-Engine nicht instabil wird. Daher muss man bei Renderern die weniger als 7 FPS haben, den Zeitunterschied zwischen Frames mithilfe der Funktion `Time.get_ticks_usec()` messen, die einem die vergangene Zeit mit der System-Uhr gibt.

Der Vergleich zwischen den Raycasting und Voxel-Ray-Traversal ist schwierig, da man Algorithmen die ein Strahl pro Pixel nutzen nie fair mit etwas wie Wireframe vergleichen kann. Würde man die gleiche Auflösung wie beim Wireframe einstellen, würden beide Algorithmen ein Frame pro mehrere Minuten haben.

Voxel-Ray-Traversal macht den Vergleich noch zusätzlich schwieriger, da man die Voxel Grösse einstellen kann. Szenen bei denen Raycasting und Wireframe unzählige kleine Flächen zeigen müssen, werden für den Voxel-Ray-Traversal zu Voxeln vereinfacht.

Der vollständige Quellcode des Projekts sowie alle nachfolgend beschriebenen Skripte sind öffentlich in einem GitHub-Repository unter https://github.com/kommeleon/matura einsehbar.

3.2 Testaufbau

3.2.1 Aufbau Szenen in Blender

Um die Rendering-Algorithmen zu testen modellierte ich zuerst eine Szene in Blender die ich dann mithilfe des Blender-Python Moduls auf die in Godot nötige Datenstruktur exportieren kann.

Insgesamt machte ich drei Szenen in Blender die verschiedene Ziele hatten.

1. Die erste Szene besteht aus 12 Objekten und ist ein Raum mit ein paar Zielscheiben, es sollte ähnlich sein zu gewöhnlichen First-Person-Shooter Räumen.
2. Die zweite hat Quaderförmige Plattformen die in der Luft schweben, um zu testen wie gut die Renderer funktionieren wenn es nur wenige simple Objekte in der Szene gibt.
3. Die dritte besteht aus Objekte mit vielen Eckpunkten, um zu testen wie gut die Renderer komplexe Objekte anzeigen können.

3.2.2 Datenstrukturen der Szenen

Für alle drei Renderer werden verschiedene Datenstrukturen benötigt, damit die Renderer funktionieren.

- Wireframe-Rendering: Pro Objekt ein Dictionary. Es hat eine Liste für alle Koordinaten der Eckpunkte und eine Liste mit den Listen von Indizes der Punkte in der Reihe, in der sie sich zu einer Fläche verbinden. Wireframe-Rendering kann Polygone anzeigen, also können die Flächen beliebig viele Punkte haben zum verbinden.
- Raycasting: Pro Objekt eine Liste die Listen mit Grösse drei enthält. Da Raycasting nur mit Dreiecken funktioniert, ist es effizienter jeden Eckpunkt mehrere Male aufzuschreiben anstatt eine weitere Liste mit

Indizes zu haben. Die Reihenfolge mit der sich die Dreiecke verbinden ist auch irrelevant, da alle Dreiecke sich immer nur auf eine Art verbinden können.

- Voxel-Ray-Traversal: Es braucht die gleiche Datenstruktur wie Raycasting, wandelt aber vor den Rendering alle Objekte in der Szene zu Voxels um. Wie genau das gemacht wird, wurde schon in Kapitel 2.3 erklärt. Die Voxels werden in einem Dictionary gespeichert, welches pro besetztem Voxel einen dreidimensionalen ganzzahligen Vektor speichert.

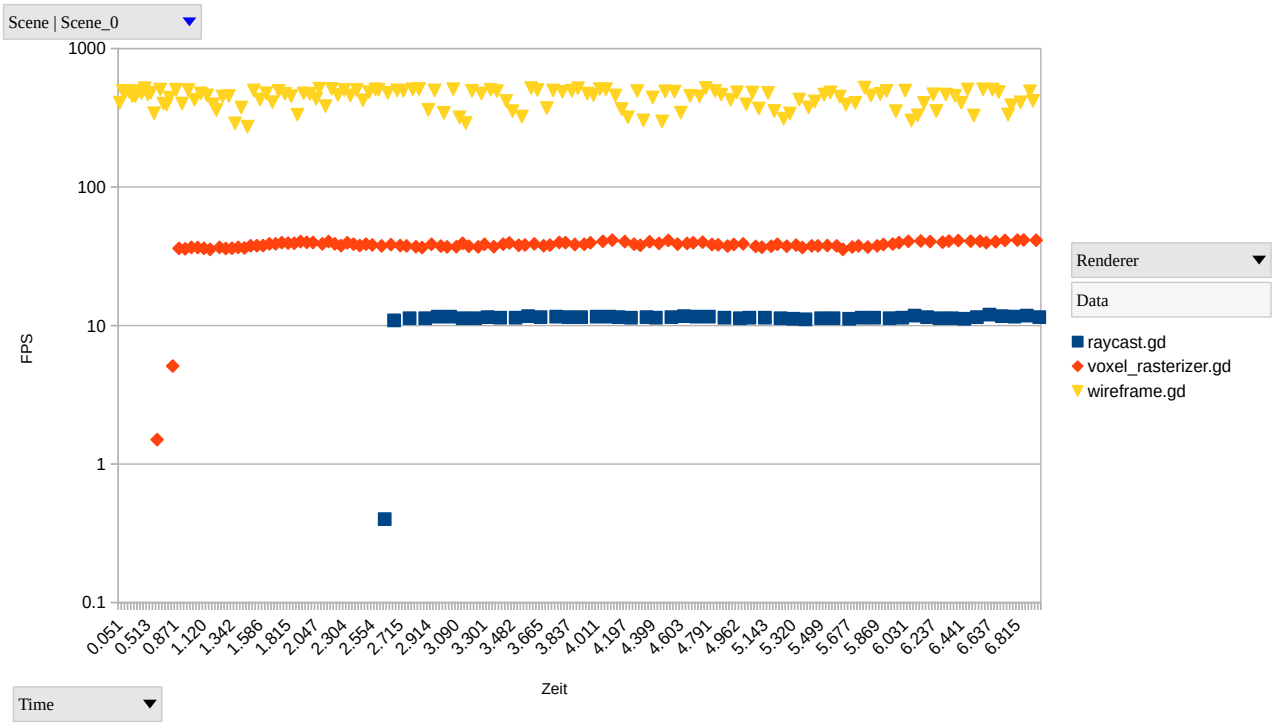
3.3 Simulation

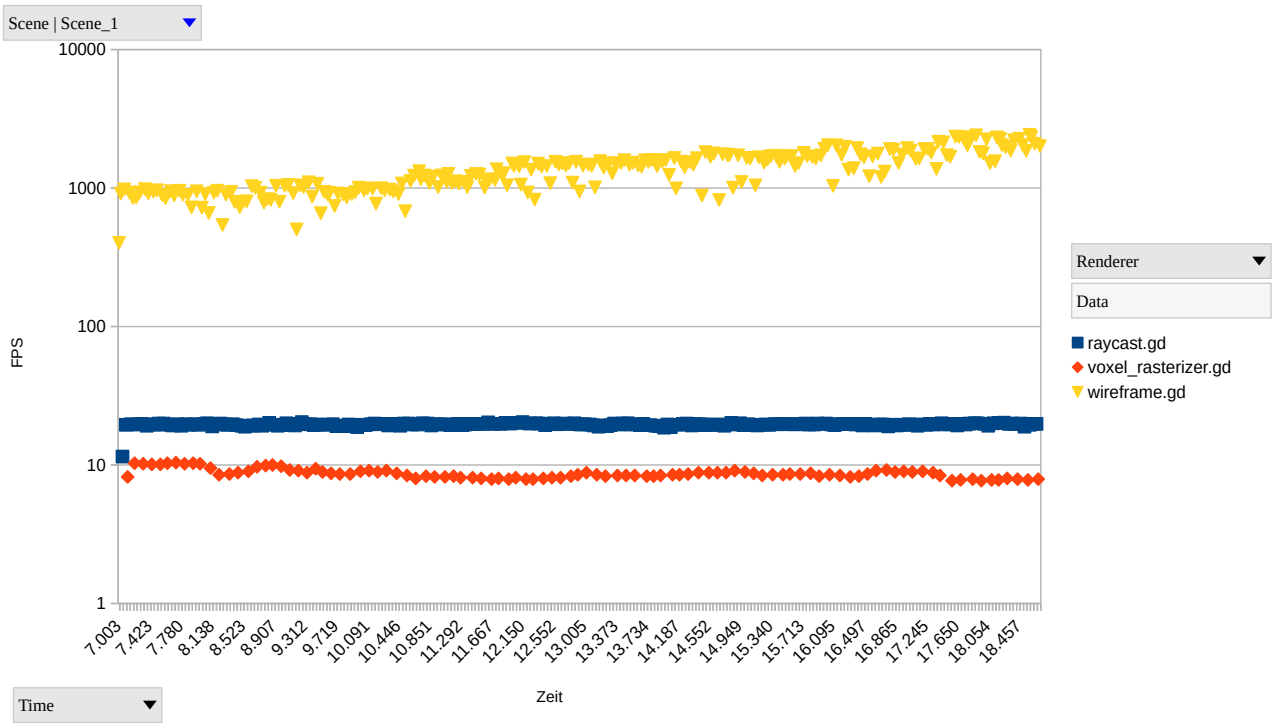
Damit bei allen drei Renderern das gleiche getestet wird baute ich in meinem Programm eine Funktion ein, die die Änderung der Kamera Bewegung und Rotation zwischen jedem erzeugten Frame aufnimmt. Es speichert ausserdem in welcher Szene man zur Zeit war.

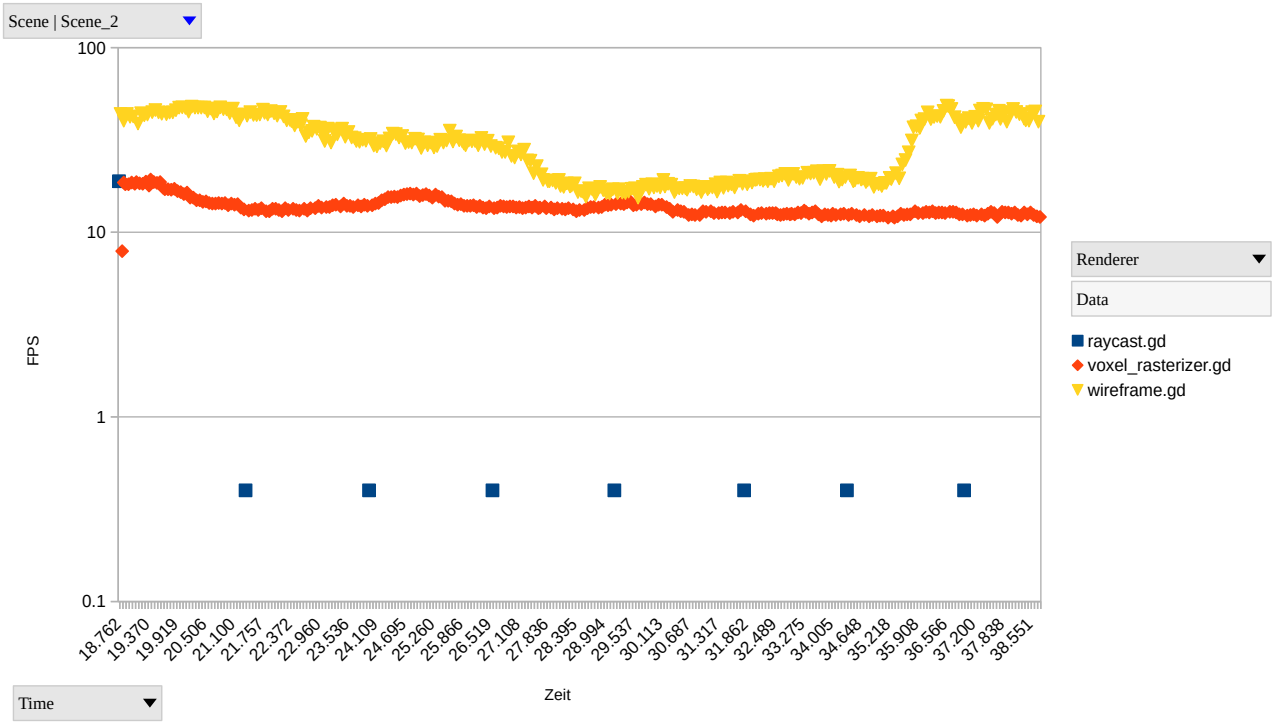
Danach kann man mit einer Taste diese Folge von Bewegung und Rotation abspielen lassen für alle drei Renderer. Dies ermöglicht einen besseren Vergleich zwischen den Renderern, da alle den gleichen Ablauf rendern müssen.

4 Ergebnisse

In den folgenden drei Diagrammen ist die gemessene Bildwiederholrate (FPS) auf einer logarithmischen Skala über die Zeit (in Sekunden) für alle drei Szenen dargestellt. Dabei entspricht Scene_0 der ersten Szene (Raum mit Zielscheiben), Scene_1 der zweiten Szene (schwebende Quader) und Scene_2 der dritten Szene (komplexe Objekte).







5 Interpretation

5.1 Vergleichende Analyse der Algorithmen

Wie schon zu erwarten war, war das Wireframe Rendering in allen Szenen den anderen Rendering-Algorithmen überlegen. Das liegt daran, dass Strahlen-basierte Algorithmen pro Bild viel mehr Berechnungen durchführen müssen, als das Wireframe-Rendering.

Auch war es zu erwarten dass der Voxel-Ray-Traversal besser als Raycasting war. Raycasting muss für jeden Strahl jedes Dreieck in jedem Objekt berechnen, ob es etwas treffen wird und wie weit es entfernt ist. Voxel-Ray-Traversal funktioniert umso besser, je näher die Voxels am Spieler sind, da die Strahlen weniger Schritte machen müssen, bis sie ein Voxel treffen. Deswegen hatte es die beste Leistung in einem geschlossen Raum, während es in der zweiten Szene, die sehr offen ist, die schlechteste Leistung zeigt und sogar schlechter als Raycasting ist. Die Strahlen müssen weit reisen bis sie die maximale Anzahl an Schritte erreichen und aufgeben.

5.2 Limitationen der Godot Implementierung

Beim Vergleich der FPS-Werte gibt es Limitationen bei der Implementierung die berücksichtigt werden müssen. Die wichtigste ist, dass alle Implementierungen in der Programmiersprache GDScript geschrieben worden sind. GDScript Code wird auf der CPU ausgeführt, was bei Algorithmen für Grafik besonders ineffizient ist. Das ist auch der Grund, wieso Computer auch eine Grafikkarte haben, die dafür gemacht ist, tausende Pixel parallel zu berechnen. Grafik-Algorithmen in modernen Videospielen werden entweder in maschinennahen Programmiersprachen wie C++ oder Rust programmiert oder über sogenannte Shader direkt auf der Grafikkarte berechnet.

Ausserdem musste aus Leistungsgründen die Bildschirmauflösung beim Raycasting und beim Voxel-Ray-Traversal arbiträr gewählt werden, da eine Auflösung 480p oder höher bei beiden Algorithmen nicht spielbar wäre.

6 Zusammenfassung und Schlusswort

Quellenverzeichnis

[1] I. E. Sutherland und G. W. Hodgman, «Reentrant Polygon Clipping», *Communications of the ACM*, Bd. 17, Nr. 1, S. 32–42, 1974.

[2] T. Möller und B. Trumbore, «Fast, Minimum Storage Ray/Triangle Intersection», *Journal of Graphics Tools*, Bd. 2, Nr. 1, S. 21–28, 1997.

[3] J. Amanatides und A. Woo, «A Fast Voxel Traversal Algorithm for Ray Tracing», in *Eurographics '87*, Amsterdam: North-Holland, 1987, S. 3–10.